

PCA20S02J - DATA ANALYSIS USING R

UNIT 2

R - Data Types

In contrast to other programming languages like C and java in R, the variables are not declared as some data type. The variables are assigned with R-Objects and the data type of the R-object becomes the data type of the variable. There are many types of R-objects. The frequently used ones are –

- Vectors
- Lists
- Matrices
- Arrays
- Factors
- Data Frames

The simplest of these objects is the vector object and there are six data types of these atomic vectors, also termed as six classes of vectors.

In R programming, the very basic data types are the R-objects called vectors which hold elements of different classes as shown above. Please note in R the number of classes is not confined to only the above six types. For example, we can use many atomic vectors and create an array whose class will become array.

1. Vectors

When you want to create vector with more than one element, you should use `c()` function which means to combine the elements into a vector.

Create a vector.

```
apple <- c('red','green',"yellow")
```

```
print(apple)
```

```
# Get the class of the vector.
```

```
print(class(apple))
```

When we execute the above code, it produces the following result –

```
[1] "red"  "green" "yellow"
```

```
[1] "character"
```

2. Lists

A list is an R-object which can contain many different types of elements inside it like vectors, functions and even another list inside it.

```
# Create a list.
```

```
list1 <- list(c(2,5,3),21.3,sin)
```

```
# Print the list.
```

```
print(list1)
```

When we execute the above code, it produces the following result –

```
[[1]]
```

```
[1] 2 5 3
```

```
[[2]]
```

```
[1] 21.3
```

```
[[3]]
```

```
function (x) .Primitive("sin")
```

3. Matrices

A matrix is a two-dimensional rectangular data set. It can be created using a vector input to the matrix function.

Create a matrix.

```
M = matrix( c('a','a','b','c','b','a'), nrow = 2, ncol = 3, byrow = TRUE)
```

```
print(M)
```

When we execute the above code, it produces the following result –

```
      [,1] [,2] [,3]  
[1,] "a"  "a"  "b"  
[2,] "c"  "b"  "a"
```

4. Arrays

While matrices are confined to two dimensions, arrays can be of any number of dimensions. The array function takes a dim attribute which creates the required number of dimension. In the below example we create an array with two elements which are 3x3 matrices each.

Create an array.

```
a <- array(c('green','yellow'),dim = c(3,3,2))
```

```
print(a)
```

When we execute the above code, it produces the following result –

```
,, 1  
      [,1] [,2] [,3]  
[1,] "green" "yellow" "green"  
[2,] "yellow" "green" "yellow"  
[3,] "green" "yellow" "green"  
  
,, 2  
      [,1] [,2] [,3]
```

```
[1,] "yellow" "green" "yellow"
```

```
[2,] "green" "yellow" "green"
```

```
[3,] "yellow" "green" "yellow"
```

Factors

Factors are the R-objects which are created using a vector. It stores the vector along with the distinct values of the elements in the vector as labels. The labels are always character irrespective of whether it is numeric or character or Boolean etc. in the input vector. They are useful in statistical modeling.

Factors are created using the `factor()` function. The `nlevels` function gives the count of levels.

```
# Create a vector.
```

```
apple_colors <- c('green','green','yellow','red','red','red','green')
```

```
# Create a factor object.
```

```
factor_apple <- factor(apple_colors)
```

```
# Print the factor.
```

```
print(factor_apple)
```

```
print(nlevels(factor_apple))
```

When we execute the above code, it produces the following result –

```
[1] green green yellow red  red  red  green
```

```
Levels: green red yellow
```

```
[1] 3
```

5. Data Frames

Data frames are tabular data objects. Unlike a matrix in data frame each column can contain different modes of data. The first column can be numeric while the second column can be character and third column can be logical. It is a list of vectors of equal length.

Data Frames are created using the data.frame() function.

Create the data frame.

```
BMI <- data.frame(
  gender = c("Male", "Male","Female"),
  height = c(152, 171.5, 165),
  weight = c(81,93, 78),
  Age = c(42,38,26)
)
print(BMI)
```

When we execute the above code, it produces the following result –

	gender	height	weight	Age
1	Male	152.0	81	42
2	Male	171.5	93	38
3	Female	165.0	78	26

R – Variables

A variable provides us with named storage that our programs can manipulate. A variable in R can store an atomic vector, group of atomic vectors or a combination of many R objects. A valid variable name consists of letters, numbers and the dot or underline characters. The variable name starts with a letter or the dot not followed by a number.

1. Variable Assignment

The variables can be assigned values using leftward, rightward and equal to operator. The values of the variables can be printed using print() or cat() function. The cat() function combines multiple items into a continuous print output.

Assignment using equal operator.

```
var.1 = c(0,1,2,3)
```

Assignment using leftward operator.

```
var.2 <- c("learn","R")
```

Assignment using rightward operator.

```
c(TRUE,1) -> var.3
```

```
print(var.1)
```

```
cat ("var.1 is ", var.1 ,"\n")
```

```
cat ("var.2 is ", var.2 ,"\n")
```

```
cat ("var.3 is ", var.3 ,"\n")
```

When we execute the above code, it produces the following result –

```
[1] 0 1 2 3
```

```
var.1 is 0 1 2 3
```

```
var.2 is learn R
```

```
var.3 is 1 1
```

2. Data Type of a Variable

In R, a variable itself is not declared of any data type, rather it gets the data type of the R - object assigned to it. So R is called a dynamically typed language, which means that we can change a variable's data type of the same variable again and again when using it in a program.

```
var_x <- "Hello"
```

```
cat("The class of var_x is ",class(var_x),"\n")
```

```
var_x <- 34.5
```

```
cat(" Now the class of var_x is ",class(var_x),"\n")
```

```
var_x <- 27L
```

```
cat(" Next the class of var_x becomes ",class(var_x),"\n")
```

When we execute the above code, it produces the following result –

```
The class of var_x is character
```

Now the class of `var_x` is `numeric`

Next the class of `var_x` becomes `integer`

3. Finding Variables

To know all the variables currently available in the workspace we use the `ls()` function. Also the `ls()` function can use patterns to match the variable names.

```
print(ls())
```

When we execute the above code, it produces the following result –

```
[1] "my var"    "my_new_var" "my_var"    "var.1"  
[5] "var.2"    "var.3"     "var.name"  "var_name2."  
[9] "var_x"    "varname"
```

The `ls()` function can use patterns to match the variable names.

List the variables starting with the pattern "var".

```
print(ls(pattern = "var"))
```

When we execute the above code, it produces the following result –

```
[1] "my var"    "my_new_var" "my_var"    "var.1"  
[5] "var.2"    "var.3"     "var.name"  "var_name2."  
[9] "var_x"    "varname"
```

The variables starting with `dot(.)` are hidden, they can be listed using `"all.names = TRUE"` argument to `ls()` function.

```
print(ls(all.name = TRUE))
```

When we execute the above code, it produces the following result –

```
[1] ".cars"     ".Random.seed" ".var_name"   ".varname"   ".varname2"  
[6] "my var"    "my_new_var"  "my_var"     "var.1"      "var.2"
```

```
[1]"var.3"      "var.name"    "var_name2." "var_x"
```

4. Deleting Variables

Variables can be deleted by using the `rm()` function. Below we delete the variable `var.3`. On printing the value of the variable error is thrown.

```
rm(var.3)
```

```
print(var.3)
```

When we execute the above code, it produces the following result –

```
[1] "var.3"
```

Error in print(var.3) : object 'var.3' not found

All the variables can be deleted by using the `rm()` and `ls()` function together.

```
rm(list = ls())
```

```
print(ls())
```

When we execute the above code, it produces the following result –

```
character(0)
```

R – Operators

An operator is a symbol that tells the compiler to perform specific mathematical or logical manipulations. R language is rich in built-in operators and provides following types of operators.

Types of Operators

We have the following types of operators in R programming –

- Arithmetic Operators
- Relational Operators
- Logical Operators
- Assignment Operators
- Miscellaneous Operators

Arithmetic Operators

Following table shows the arithmetic operators supported by R language. The operators act on each element of the vector.

Operator	Description	Example
+	Adds two vectors	<pre>v <- c(2,5.5,6) t <- c(8, 3, 4) print(v+t) it produces the following result – [1] 10.0 8.5 10.0</pre>
-	Subtracts second vector from the first	<pre>v <- c(2,5.5,6) t <- c(8, 3, 4) print(v-t) it produces the following result – [1] -6.0 2.5 2.0</pre>
*	Multiplies both vectors	<pre>v <- c(2,5.5,6) t <- c(8, 3, 4) print(v*t) it produces the following result – [1] 16.0 16.5 24.0</pre>
/	Divide the first vector with the second	<pre>v <- c(2,5.5,6) t <- c(8, 3, 4) print(v/t) When we execute the above code, it produces the following result – [1] 0.250000 1.833333 1.500000</pre>
%%	Give the remainder of the first vector with the second	<pre>v <- c(2,5.5,6) t <- c(8, 3, 4) print(v%%t) it produces the following result – [1] 2.0 2.5 2.0</pre>
/%	The result of division of first vector with second (quotient)	<pre>v <- c(2,5.5,6) t <- c(8, 3, 4) print(v/%t) it produces the following result – [1] 0 1 1</pre>

^	The first vector raised to the exponent of second vector	<pre>v <- c(2,5.5,6) t <- c(8, 3, 4) print(v^t)</pre> it produces the following result – <pre>[1] 256.000 166.375 1296.000</pre>
---	--	---

Relational Operators

Following table shows the relational operators supported by R language. Each element of the first vector is compared with the corresponding element of the second vector. The result of comparison is a Boolean value.

Operator	Description	Example
>	Checks if each element of the first vector is greater than the corresponding element of the second vector.	<pre>v <- c(2,5.5,6,9) t <- c(8,2.5,14,9) print(v>t)</pre> it produces the following result – <pre>[1] FALSE TRUE FALSE FALSE</pre>
<	Checks if each element of the first vector is less than the corresponding element of the second vector.	<pre>v <- c(2,5.5,6,9) t <- c(8,2.5,14,9) print(v < t)</pre> it produces the following result – <pre>[1] TRUE FALSE TRUE FALSE</pre>
==	Checks if each element of the first vector is equal to the corresponding element of the second vector.	<pre>v <- c(2,5.5,6,9) t <- c(8,2.5,14,9) print(v == t)</pre> it produces the following result – <pre>[1] FALSE FALSE FALSE TRUE</pre>
<=	Checks if each element of the first vector is less than or equal to the corresponding element of the second vector.	<pre>v <- c(2,5.5,6,9) t <- c(8,2.5,14,9) print(v<=t)</pre> it produces the following result – <pre>[1] TRUE FALSE TRUE TRUE</pre>
>=	Checks if each element of the first vector is greater than or equal to the corresponding element of the second vector.	<pre>v <- c(2,5.5,6,9) t <- c(8,2.5,14,9) print(v>=t)</pre> it produces the following result – <pre>[1] FALSE TRUE FALSE TRUE</pre>

!=	Checks if each element of the first vector is unequal to the corresponding element of the second vector.	<pre>v <- c(2,5.5,6,9) t <- c(8,2.5,14,9) print(v!=t)</pre> it produces the following result – <pre>[1] TRUE TRUE TRUE FALSE</pre>
----	--	---

Logical Operators

Following table shows the logical operators supported by R language. It is applicable only to vectors of type logical, numeric or complex. All numbers greater than 1 are considered as logical value TRUE.

Each element of the first vector is compared with the corresponding element of the second vector. The result of comparison is a Boolean value.

Operator	Description	Example
&	It is called Element-wise Logical AND operator. It combines each element of the first vector with the corresponding element of the second vector and gives a output TRUE if both the elements are TRUE.	<pre>v <- c(3,1,TRUE,2+3i) t <- c(4,1,FALSE,2+3i) print(v&t)</pre> it produces the following result – <pre>[1] TRUE TRUE FALSE TRUE</pre>
	It is called Element-wise Logical OR operator. It combines each element of the first vector with the corresponding element of the second vector and gives a output TRUE if one the elements is TRUE.	<pre><- c(3,0,TRUE,2+2i) t <- c(4,0,FALSE,2+3i) print(v t)</pre> it produces the following result – <pre>[1] TRUE FALSE TRUE TRUE</pre>
!	It is called Logical NOT operator. Takes each element of the vector and gives the opposite logical value.	<pre>v <- c(3,0,TRUE,2+2i) print(!v)</pre> it produces the following result – <pre>[1] FALSE TRUE FALSE FALSE</pre>
The logical operator && and considers only the first element of the vectors and give a vector of single element as output.		
&&	Called Logical AND operator. Takes first element of both the	<pre>v <- c(3,0,TRUE,2+2i) t <- c(1,3,TRUE,2+3i)</pre>

	vectors and gives the TRUE only if both are TRUE.	print(v&& t) it produces the following result – [1] TRUE
	Called Logical OR operator. Takes first element of both the vectors and gives the TRUE if one of them is TRUE.	v <- c(0,0,TRUE,2+2i) t <- c(0,3,TRUE,2+3i) print(v t) it produces the following result – [1] FALSE

R - Decision making

Decision making structures require the programmer to specify one or more conditions to be evaluated or tested by the program, along with a statement or statements to be executed if the condition is determined to be true, and optionally, other statements to be executed if the condition is determined to be false.

1. R - If Statement

An if statement consists of a Boolean expression followed by one or more statements.

The basic syntax for creating an if statement in R is –

if(boolean_expression)

{

// statement(s) will execute if the boolean expression is true.

}

If the Boolean expression evaluates to be true, then the block of code inside the if statement will be executed. If Boolean expression evaluates to be false, then the first set of code after the end of the if statement (after the closing curly brace) will be executed.

Example

```
x <- 30L
```

```
if(is.integer(x)) {  
  
  print("X is an Integer")  
  
}
```

When the above code is compiled and executed, it produces the following result –

```
[1] "X is an Integer"
```

2. R - If...Else Statement

An if statement can be followed by an optional else statement which executes when the boolean expression is false.

The basic syntax for creating an if...else statement in R is –

```
if(boolean_expression) {  
  
  // statement(s) will execute if the boolean expression is true.  
  
} else {  
  
  // statement(s) will execute if the boolean expression is false.  
  
}
```

Example

```
x <- c("what","is","truth")

if("Truth" %in% x) {

  print("Truth is found")

} else {

  print("Truth is not found")

}
```

When the above code is compiled and executed, it produces the following result –

```
[1] "Truth is not found"
```

Here "Truth" and "truth" are two different strings.

3. The if...else if...else Statement

An if statement can be followed by an optional else if...else statement, which is very useful to test various conditions using single if...else if statement.

When using if, else if, else statements there are few points to keep in mind.

- An if can have zero or one else and it must come after any else if's.
- An if can have zero to many else if's and they must come before the else.
- Once an else if succeeds, none of the remaining else if's or else's will be tested.

The basic syntax for creating an if...else if...else statement in R is –

```
if(boolean_expression 1) {

  // Executes when the boolean expression 1 is true.

} else if( boolean_expression 2) {
```

```

    // Executes when the boolean expression 2 is true.

} else if( boolean_expression 3) {

    // Executes when the boolean expression 3 is true.

} else {

    // executes when none of the above condition is true.

}

```

Example

```

x <- c("what","is","truth")

if("Truth" %in% x) {

    print("Truth is found the first time")

} else if ("truth" %in% x) {

    print("truth is found the second time")

} else {

    print("No truth found")

}

```

When the above code is compiled and executed, it produces the following result –

```
[1] "truth is found the second time"
```

4. R - Switch Statement

A switch statement allows a variable to be tested for equality against a list of values. Each value is called a case, and the variable being switched on is checked for each case.

The basic syntax for creating a switch statement in R is –

switch(expression, case1, case2, case3....)

The following rules apply to a switch statement –

- If the value of expression is not a character string it is coerced to integer.
- You can have any number of case statements within a switch. Each case is followed by the value to be compared to and a colon.
- If the value of the integer is between 1 and nargs()-1 (The max number of arguments) then the corresponding element of case condition is evaluated and the result returned.
- If expression evaluates to a character string then that string is matched (exactly) to the names of the elements.
- If there is more than one match, the first matching element is returned.
- No Default argument is available.

In the case of no match, if there is a unnamed element of ... its value is returned. (If there is more than one such argument an error is returned.)

Example

```
x <- switch(  
  
  3,  
  
  "first",  
  
  "second",  
  
  "third",  
  
  "fourth"  
)
```



```
print(x)
```

When the above code is compiled and executed, it produces the following result –

```
[1] "third"
```

R – Loops

A loop statement allows us to execute a statement or group of statements multiple times and the following is the general form of a loop statement in most of the programming languages.

A loop statement allows us to execute a statement or group of statements multiple times and the following is the general form of a loop statement in most of the programming languages.

1. R - Repeat Loop

The Repeat loop executes the same code again and again until a stop condition is met.

The basic syntax for creating a repeat loop in R is –

```
repeat {  
  
  commands  
  
  if(condition) {  
  
    break  
  
  }  
  
}
```

Example

```
v <- c("Hello","loop")  
  
cnt <- 2
```

```
repeat {  
  
  print(v)  
  
  cnt <- cnt+1  
  
  if(cnt > 5) {  
  
    break  
  
  }  
  
}
```

When the above code is compiled and executed, it produces the following result –

```
[1] "Hello" "loop"
```

```
[1] "Hello" "loop"
```

```
[1] "Hello" "loop"
```

```
[1] "Hello" "loop"
```

2. R - While Loop

The While loop executes the same code again and again until a stop condition is met.

The basic syntax for creating a while loop in R is –

```
while (test_expression)  
  
{  
  
  statement  
  
}
```

Here key point of the while loop is that the loop might not ever run. When the condition is tested and the result is false, the loop body will be skipped and the first statement after the while loop will be executed.

Example

```
v <- c("Hello","while loop")
```

```
cnt <- 2
```

```
while (cnt < 7) {
```

```
  print(v)
```

```
  cnt = cnt + 1
```

```
}
```

When the above code is compiled and executed, it produces the following result –

```
[1] "Hello" "while loop"
```

```
[1] "Hello" "while loop"
```

```
[1] "Hello" "while loop"
```

```
[1] "Hello" "while loop"
```

```
[1] "Hello" "while loop"
```

3. R - For Loop

A For loop is a repetition control structure that allows you to efficiently write a loop that needs to execute a specific number of times.

The basic syntax for creating a for loop statement in R is –

```
for (value in vector)
```

```
{  
  
  statements  
  
}
```

R's for loops are particularly flexible in that they are not limited to integers, or even numbers in the input. We can pass character vectors, logical vectors, lists or expressions.

Example

```
v <- LETTERS[1:4]  
  
for ( i in v) {  
  
  print(i)  
  
}
```

When the above code is compiled and executed, it produces the following result –

```
[1] "A"
```

```
[1] "B"
```

```
[1] "C"
```

```
[1] "D"
```